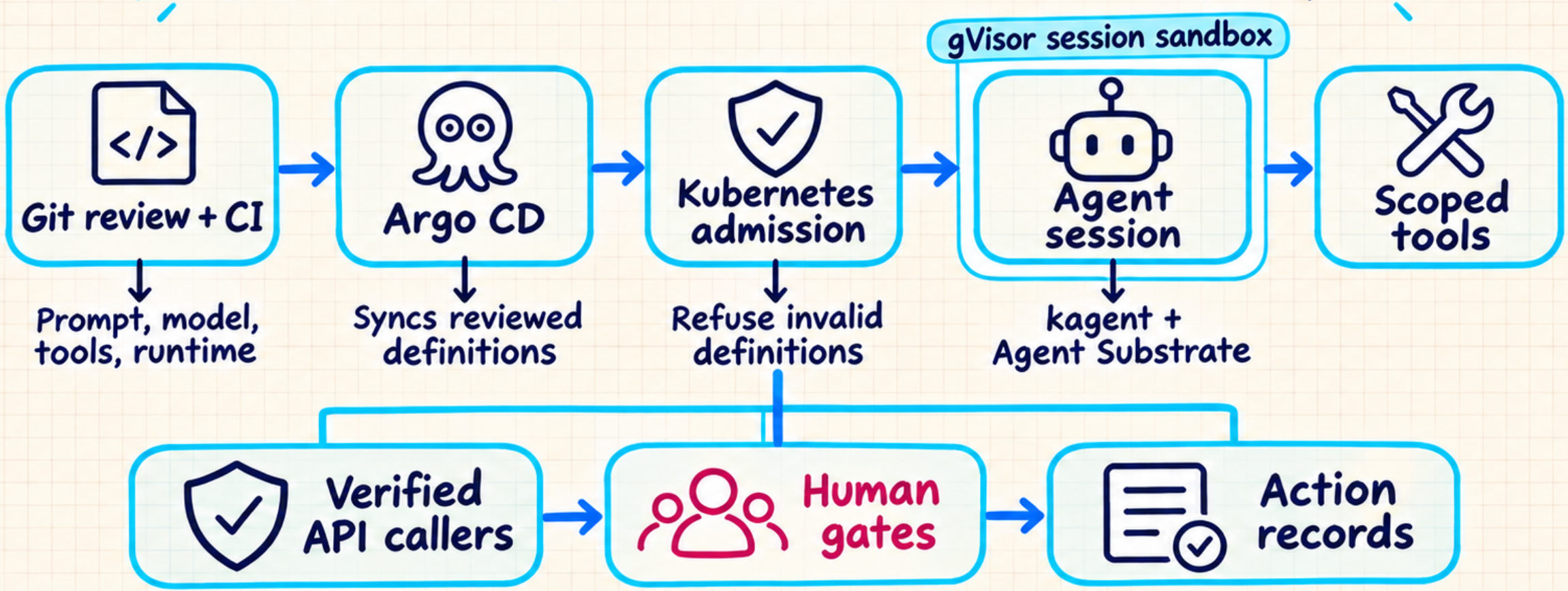
HOW I DEPLOY AGENTS SAFELY

Useful autonomy, with boundaries outside the prompt.



Change the definition in Git. Constrain what can run.

Configured controls; live health not checked. Runtime remains alpha.

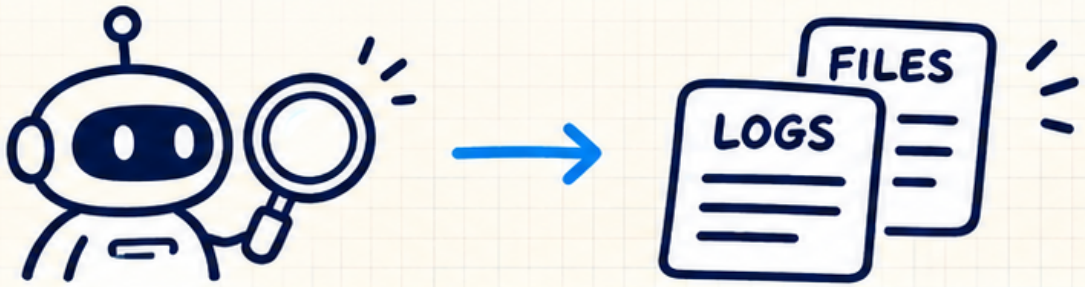
THREE LEVELS OF AUTONOMY

The tool list sets the scope; the prompt explains the job.

1 READ

Inspect and explain

k8s-reader · bug-investigator



2 PROPOSE

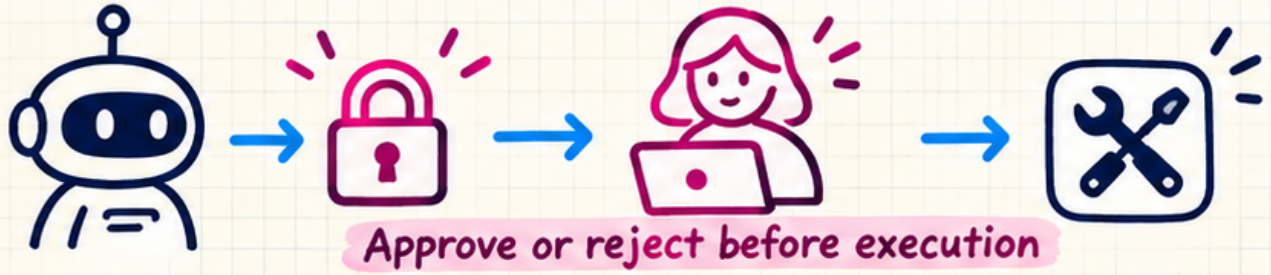
Create a branch + PR

implementer



3 ACT WITH APPROVAL

Write / destructive call



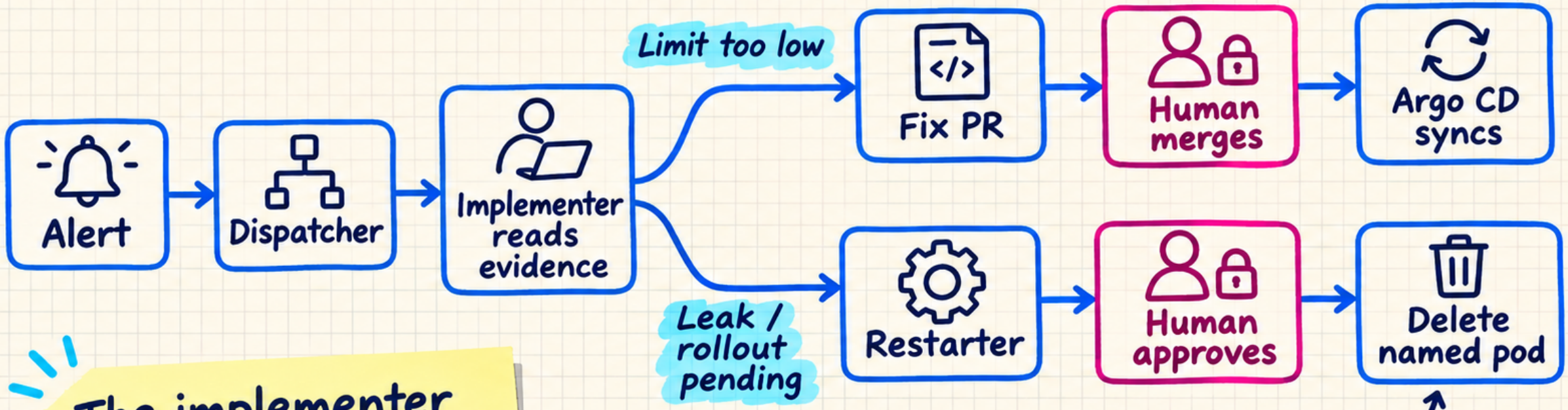
Proposal tools are ungated; merge is the human gate.

Shared Kubernetes tool credentials remain broad; direct Secret API access is excluded.

Named tools only • Unknown tools denied • Ungated mutations denied

A FIX HAS TWO DIFFERENT GATES

Example: an alert starts an investigation.

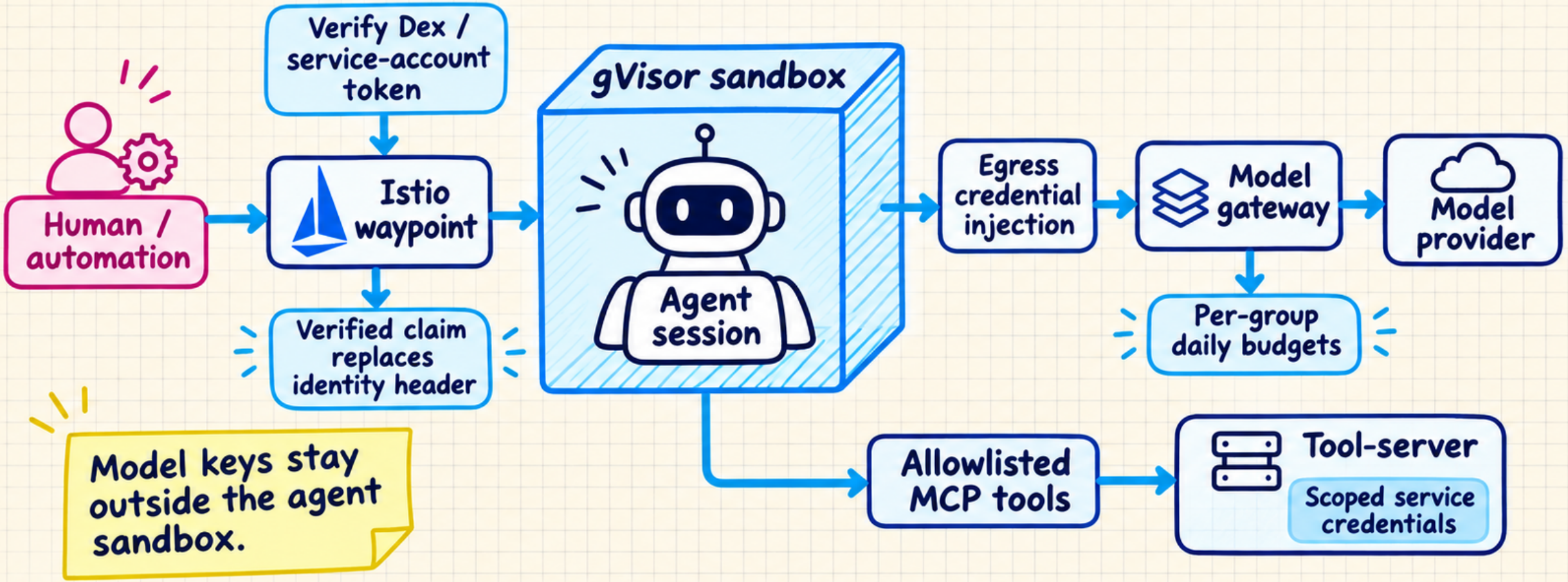


The implementer proposes a change. The restarter waits for a decision.

Inconclusive or contention: report evidence, no fix PR.

BOUNDARIES AROUND EVERY SESSION

Identity, isolation and credentials are separate controls.



⚠ UI shares an identity • Identity override gap remains • Direct in-cluster paths remain

TEST THE AGENT. WATCH THE GUARDRAILS.

A good prompt is only one part of the safety story.

AFTER A DEFINITION CHANGE

Eval gate

- ☑ Expected + forbidden tool calls
- ☑ Injection / refusal tests
- ☑ Fake-secret leak canaries

→



Blocks new Dispatcher + Slack sessions

PostSync check; not a pre-deploy block

DURING OPERATION



Calls + approval decisions → Audit + alerts



Budget exhausted → model calls blocked

Constrain actions, detect regressions, keep humans at the consequential gates.

⚠ **Limits:** CLI/UI bypass eval holds; missing hold file is unknown. Shared UI identity and alpha runtime still need work.